

Start testing

Start testing

1. Course Content
2. Calibrate the car's centerline
3. Calibrate lane reference lines
 - 3.1 Calibrate the Left Lane Reference Line
 - 3.2 Calibrating the Right Lane Reference Line
4. Calibration Parameter File
5. Pre-Test Preparation
 - 5.1 Calibrate Zero Bias
 - 5.2 Preview after calibration
6. Lane Line Detection Principle and Source Code Analysis
 - 6.1 Lane Line Detection Principle
 - 6.2 Lane Line Detection Source Code
 - 6.2.1 YOLOv11 Model Inference Source Code
 - 6.2.2 Main Program for Lane Line Detection Processing

1. Course Content

[!NOTE]

- This tutorial is only for users who want to optimize and debug the autonomous driving function, and to calibrate it according to their own actual situation.
- **The factory default settings do not require calibration!!
- The visual autonomous driving function uses the RGB image from the camera and does not rely on depth information.
- Using different cameras, the position of the vehicle's centerline in the camera's view will be different, so it is necessary to calibrate the vehicle's centerline in different camera views.

2. Calibrate the car's centerline

- Launch the camera node

```
ros2 launch ascamera hp60c.launch.py
```

- Launch the centerline calibration program

```
ros2 run auto_drive cali_midline
```

- Adjust the camera to a normal operating position (camera facing forward, camera angle can be slightly lower)
- Then find any reference line (any relatively straight line will do; here, the map edge line is used as the reference line), and align the center of the car body with the reference line.
- In the camera's view, we can see the reference line, which is the position of the vehicle's centerline in the current camera view. The `current mode` in the upper left corner shows the

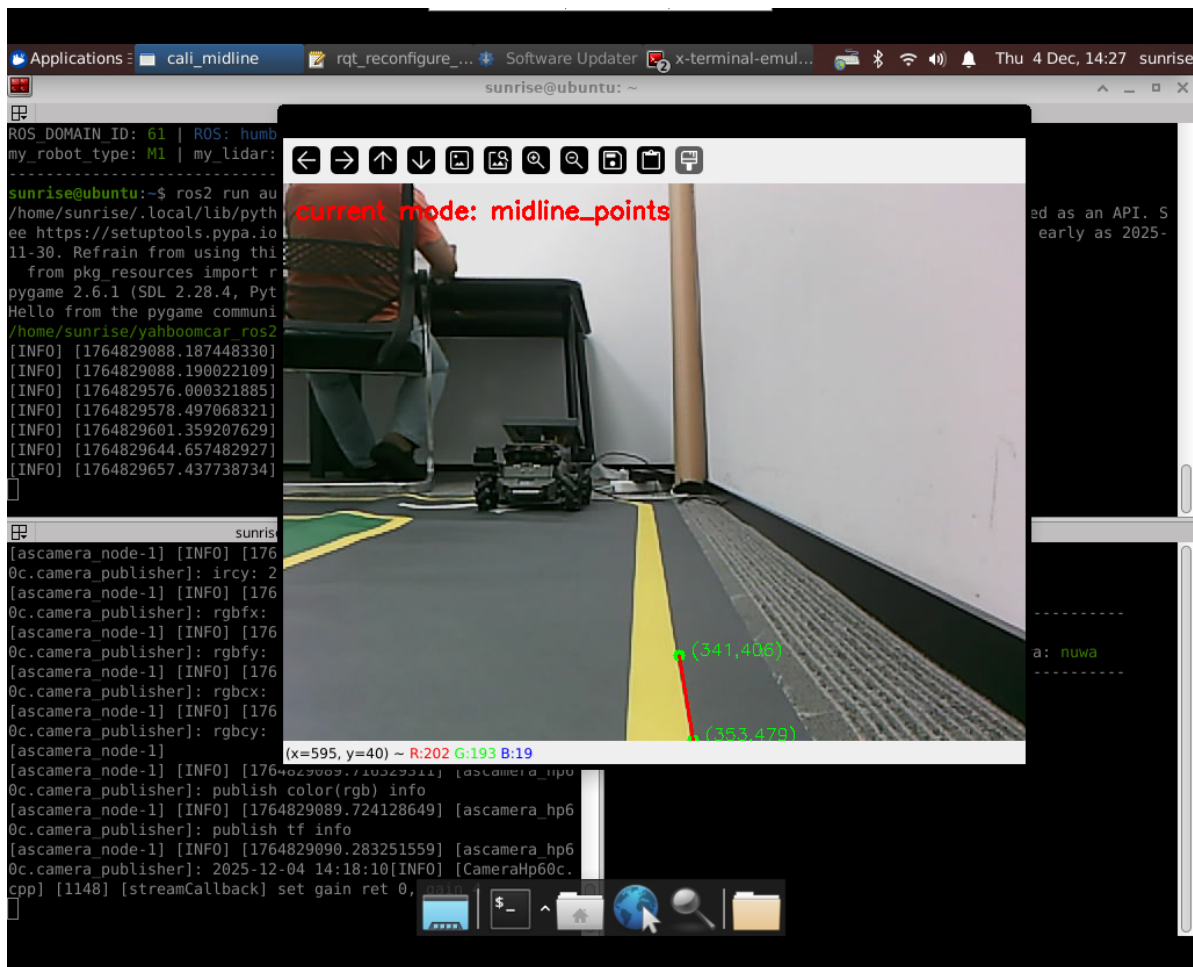
current calibration mode, and `midline_points` indicates the calibration of the vehicle's centerline.

[!NOTE]

Different cameras will make the vehicle's center line appear different due to the different camera positions. It can be seen that the vehicle's center line is not completely vertical in the camera's view; it is actually a diagonal line.

Use the mouse to select the start and end points of the central axis in sequence. Choose any two points on the reference line from bottom to top; the length doesn't need to be considered, as this can be adjusted later using parameters. After selection, the terminal will print the coordinates of the selected start and end points.

- If you are not satisfied with the calibration results or accidentally clicked the wrong button, you can press the 'r' key on the keyboard to reset the calibration state and recalibrate.



- Then press the S key on your keyboard to save. The terminal will then display the file location where the parameters are saved.

```
sunrise@ubuntu:~$ ros2 run auto_drive cali_midline
/home/sunrise/.local/lib/python3.10/site-packages/pygame/pkgdata.py:25: UserWarning: pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-
11-30. Refrain from using this package or pin to Setuptools<81.
  from pkg_resources import resource_stream, resource_exists
pygame 2.6.1 (SDL 2.28.4, Python 3.10.12)
Hello from the pygame community. https://www.pygame.org/contribute.html
/home/sunrise/yahboomcar_ws/yahboomcar_ws/install/auto_drive/share/auto_drive/config/midline_points.yaml
[INFO] [1764829088.187448330] [cali_midline]: open the cali_midline config file:cali_midline started.
[INFO] [1764829088.190022109] [cali_midline]: Press L for left line, M for middle line, R for right line
[INFO] [1764829576.000321885] [cali_midline]: Selected point: (357, 476)
[INFO] [1764829578.497068321] [cali_midline]: Selected point: (341, 406)
[INFO] [1764829601.359207629] [cali_midline]: Reset!, you can select the line segment again.
[INFO] [1764829644.657482927] [cali_midline]: Selected point: (353, 479)
[INFO] [1764829657.437738734] [cali_midline]: Selected point: (341, 406)
[INFO] [1764830008.593464664] [cali_midline]: Saved midline points to YAML file :/home/sunrise/yahboomcar_ws/yahboomcar_
ws/install/auto_drive/share/auto_drive/config/midline_points.yaml
```

[!CAUTION]

- The file save path
`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/install/auto_drive/share/auto_drive/config/midline_points.yaml` indicated in the terminal is the install directory of the `auto_drive` package.
- You need to copy the contents of this file to
`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/midline_points.yaml`.

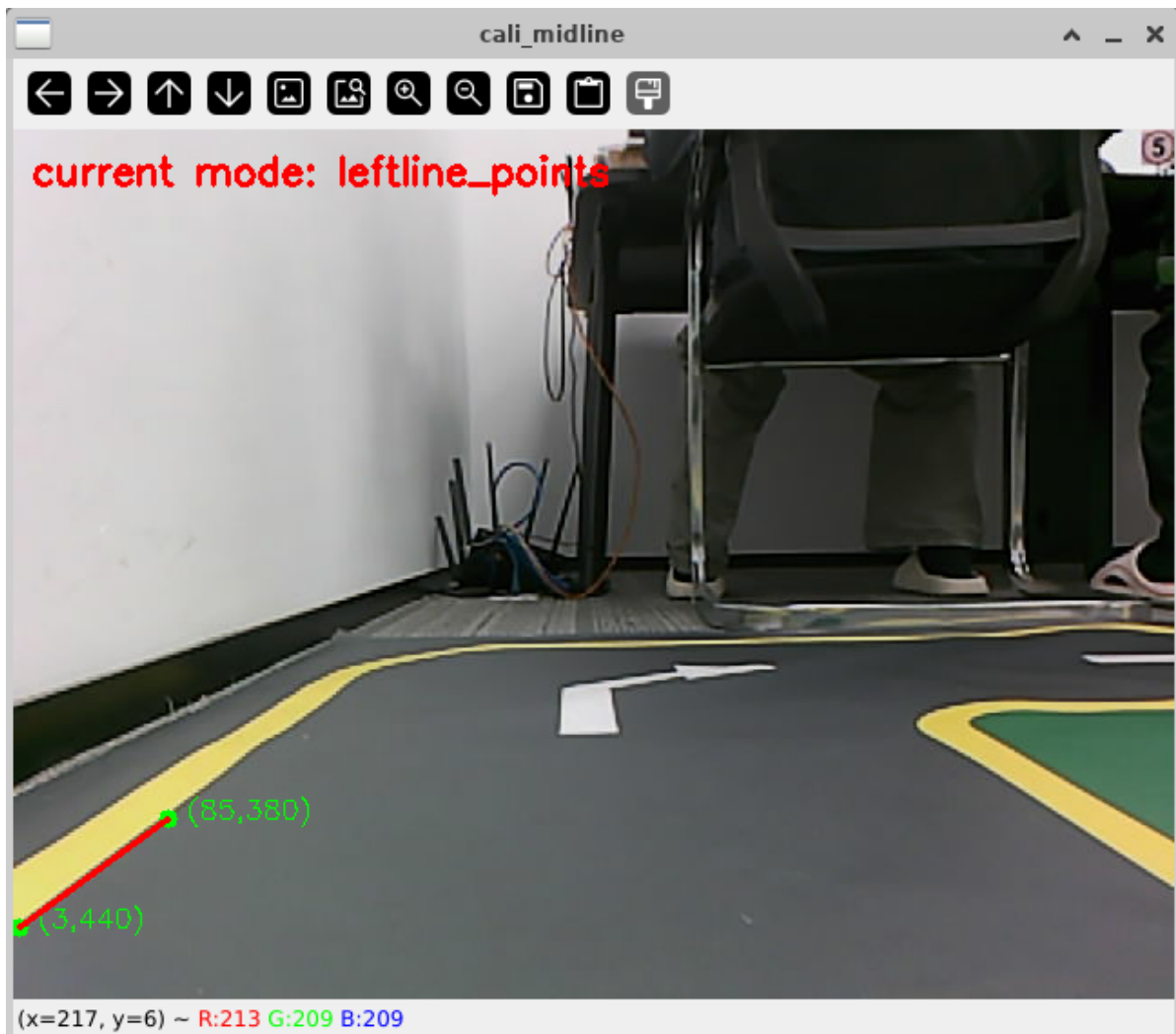
3. Calibrate lane reference lines

Place the vehicle's infotainment system in the lane on the sandbox map.

- The vehicle's centerline should be as close as possible to the center line of the lane.
- The distance between the vehicle and the left and right lane lines should be as equal as possible.

3.1 Calibrate the Left Lane Reference Line

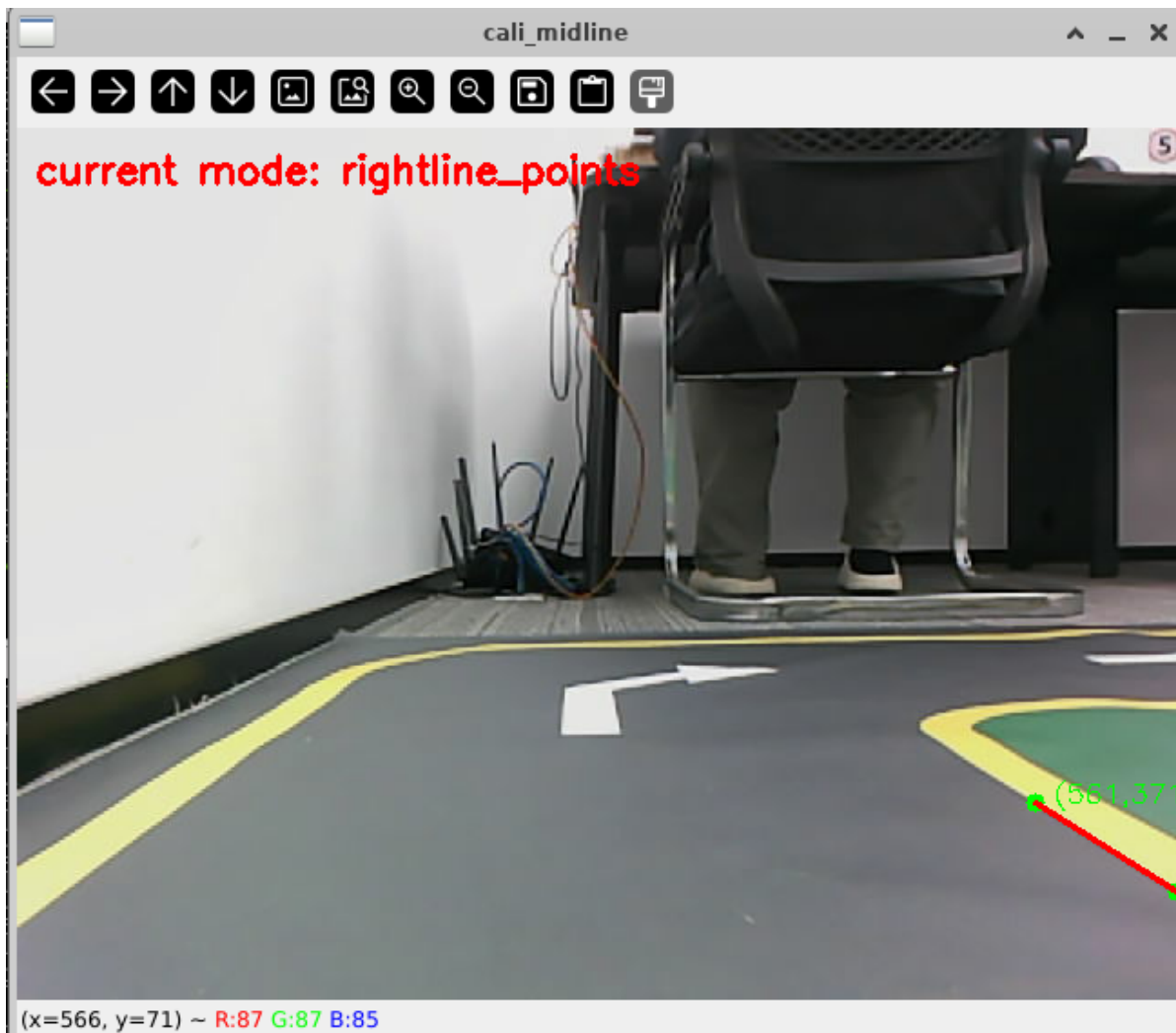
- Click the `cal_midline` heading, press the `R` key to reset the calibration status, then press the `L` key to switch to the `leftline_points` calibration mode.



- The calibration method is similar to before. Select the start and end points on the inner line of the left lane, then press the `S` key to save. The terminal will print a prompt message.

3.2 Calibrating the Right Lane Reference Line

First, press the `R` key on the keyboard to reset the calibration status, then press the `K` key to enter the `rightline_points` calibration mode.



- The calibration method is similar to the previous one. Select the start and end points on the inner line of the right lane, then press the `S` key to save. The terminal will print a prompt message.

[!TIP]

It can be observed that when the vehicle is in the middle of the lane, the left and right lane lines are actually two line segments extending inwards from the camera's perspective. Furthermore, the degree of inclination of the lane lines will differ depending on the camera's perspective.

4. Calibration Parameter File

[!CAUTION]

- The file save path
`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/install/auto_drive/share/auto_drive/config/midline_points.yaml` indicated by the terminal is the install directory of the `auto_drive` package.
- **You need to copy the contents of this file to**
`/home/sunrise/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/midline_points.yaml`

- The `midline_points` field contains the data for the vehicle's centerline.
- The `leftline_points` field is the reference line for the left lane from the camera's perspective.
- The `rightline_points` field is the reference line for the right lane from the camera's perspective.
- The `midline_length` field is the length of the vehicle's centerline (generally, the default value is sufficient; parameter tuning will be explained in later tutorials).

```
midline_points:
  start_point:
    x: 353
    y: 479
  end_point:
    x: 341
    y: 406
  mid_distance:
    - 200
    - 40
leftline_points:
  start_point:
    x: 3
    y: 440
  end_point:
    x: 85
    y: 380
rightline_points:
  start_point:
    x: 639
    y: 420
  end_point:
    x: 561
    y: 371
midline_length: 85
```

The calibration is now complete. After modifying the parameters and recompiling, the modified parameters will be used on the next run.

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
colcon build --packages-select auto_drive
```

5. Pre-Test Preparation

[!IMPORTANT]

Note: This program runs on a sandbox map. For personal maps, you will need to adjust the program yourself. This course only debugs the visualization of the car's lane line recognition.

Basic: Start the autonomous driving program and observe the lane line recognition results.

Advanced: Master the lane line detection principle using the debugging mode.

- Adjusting Test Parameters

Parameter

Path: `~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/auto_drive_node.yaml`

```
auto_drive:
  ros__parameters:
    turn_radius: 0.4
    drive_speed: 0.15
    angular_speed: -0.42
    duration: 2.0
    response_distance_1: 2900
    response_distance_2: 132
    cooldown_time: 3.0
    DEBUG_MODE: False
    START_AutoDrive: True
    combine_nav: False
    Kp: -0.4
    Ki: 0.0
    Kd: -0.1
    Kp_1: -0.3
    Ki_1: 0.0
    Kd_1: 0.0
    max_integral: 1000.0
    min_error_angular: 0.03
    offset: 10.18
    initial_error: 2.55
    image_size: [640, 480]
```

In this test, you need to change:

`START_AutoDrive`: to False. This is the master switch for the car's movement. Setting it to True will enable car movement.

`combine_nav`: to False. This doesn't need to be enabled if road network navigation isn't used.

After modifying the parameters, recompile. The next run will use the modified parameters.

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
colcon build --packages-select auto_drive
```

5.1 Calibrate Zero Bias

[!NOTE]

Please keep the car centered in the lane on the sandbox map!

Autonomous driving source code path:

`~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py`

- Start the camera node

```
ros2 launch ascamera hp60c.launch.py
```

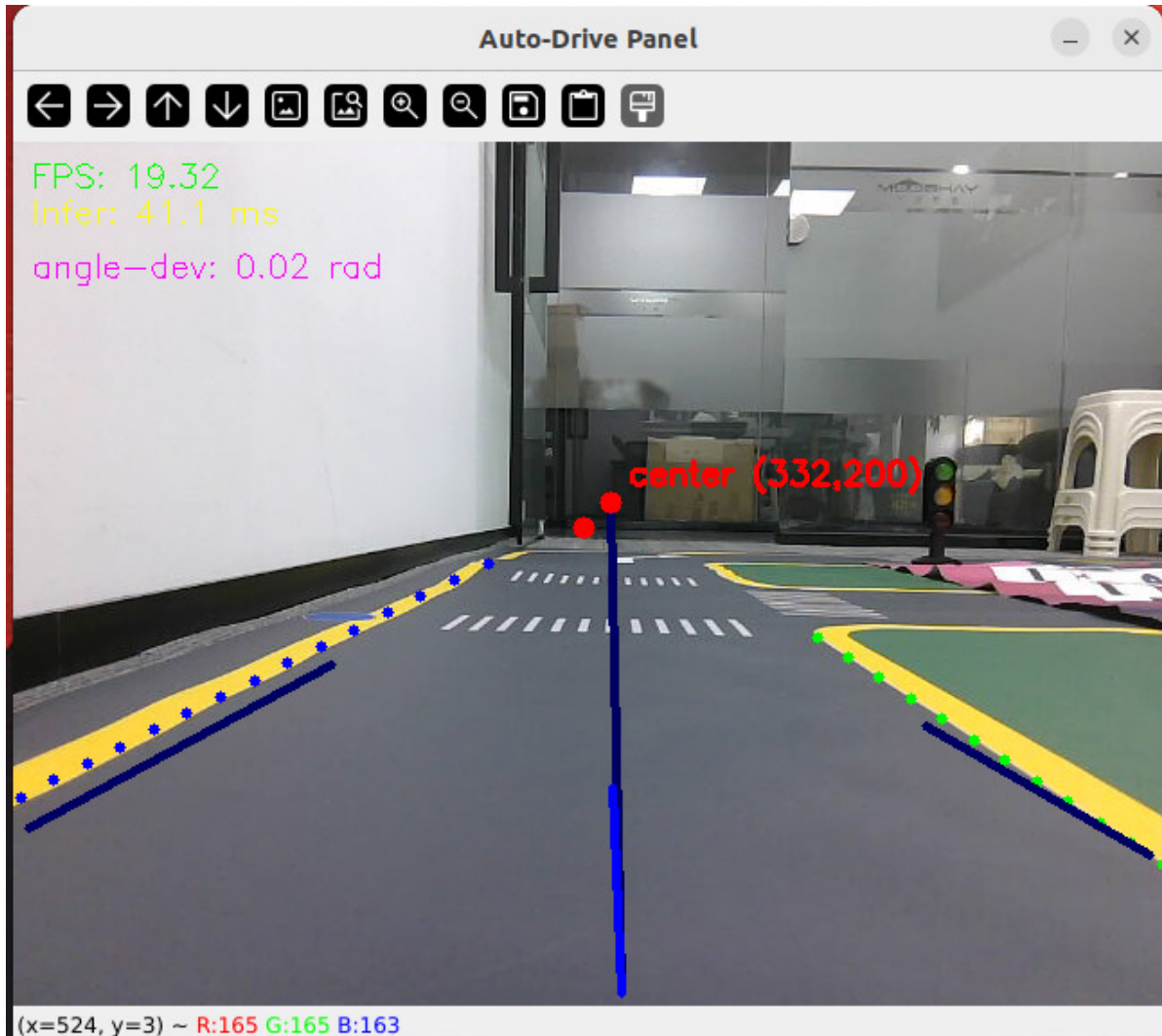
- Start the chassis

```
ros2 run yahboomcar_bringup Ackman_driver_A1
```

- Start lane line centering

```
ros2 run auto_drive auto_drive
```

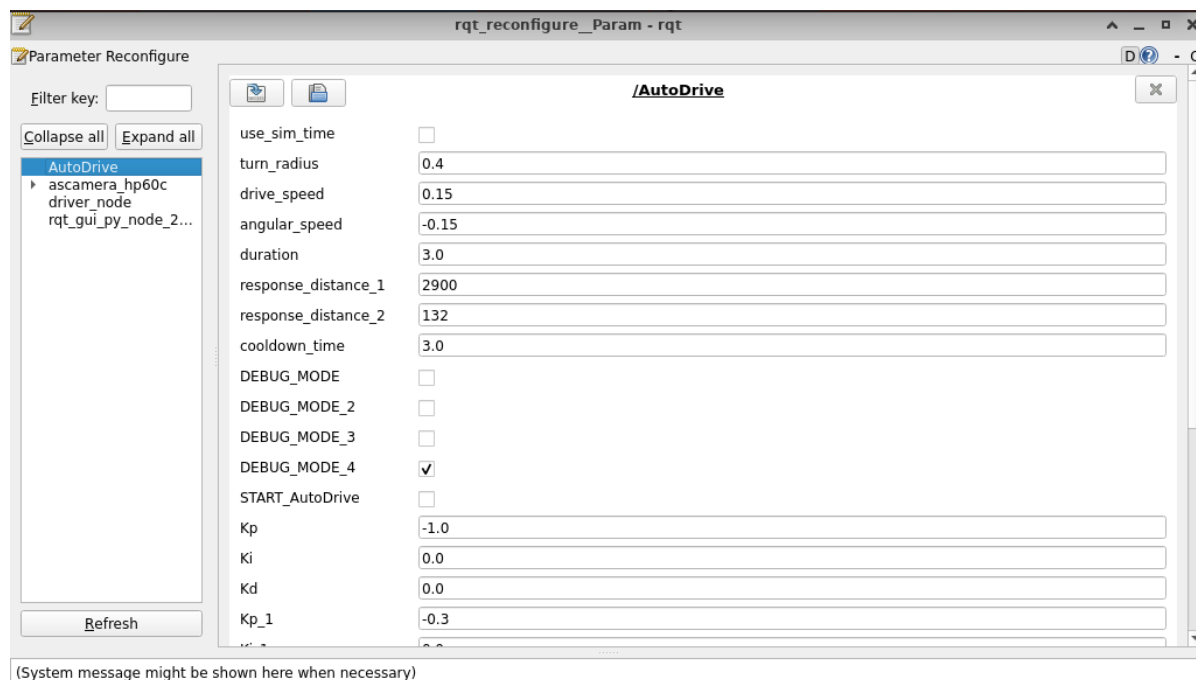
After successful startup, the screen will remain still. This is mainly for testing lane line recognition.



If you can see the image above, it means the model and program are functioning correctly. The blue dots represent the left side of the car, and the yellow dots represent the right side. The blue dot in the middle represents the angle between the centerline and its extension. Once you ensure the yellow lines on the road are recognized, you can proceed to the next step of debugging.

- Enable the dynamic parameter adjuster. The RDK master controller can execute the following commands on the virtual machine:

```
ros2 run rqt_reconfigure rqt_reconfigure
```

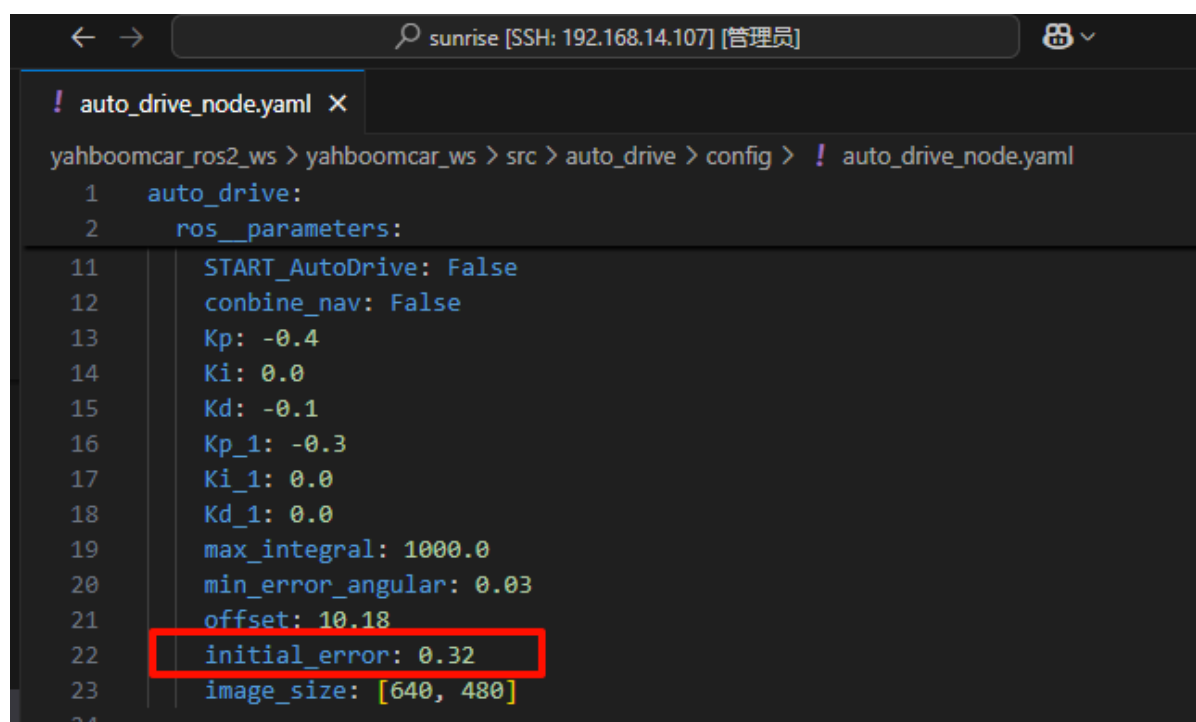



Enable `DEBUG_MODE_3`, and obtain the center-aligned zero-bias value `initial_error` in the terminal.

```
[INFO] [1764831848.541721397] [AutoDrive]: Left and right centering deviation: 0.82
[INFO] [1764831848.544512219] [AutoDrive]: Collecting initial error sample 93/100: 0.82
[INFO] [1764831848.658573403] [AutoDrive]: Left and right centering deviation: 0.59
[INFO] [1764831848.661237851] [AutoDrive]: Collecting initial error sample 94/100: 0.59
[INFO] [1764831848.738606718] [AutoDrive]: Left and right centering deviation: 0.41
[INFO] [1764831848.741164042] [AutoDrive]: Collecting initial error sample 95/100: 0.41
[INFO] [1764831848.816738875] [AutoDrive]: Left and right centering deviation: 1.19
[INFO] [1764831848.819309156] [AutoDrive]: Collecting initial error sample 96/100: 1.19
[INFO] [1764831848.898768682] [AutoDrive]: Left and right centering deviation: 0.76
[INFO] [1764831848.901156381] [AutoDrive]: Collecting initial error sample 97/100: 0.76
[INFO] [1764831848.980488616] [AutoDrive]: Left and right centering deviation: 0.17
[INFO] [1764831848.982885190] [AutoDrive]: Collecting initial error sample 98/100: 0.17
[INFO] [1764831849.104528220] [AutoDrive]: Left and right centering deviation: 1.09
[INFO] [1764831849.107374834] [AutoDrive]: Collecting initial error sample 99/100: 1.09
[INFO] [1764831849.190644512] [AutoDrive]: Left and right centering deviation: 0.75
[INFO] [1764831849.193540876] [AutoDrive]: Collecting initial error sample 100/100: 0.75
[INFO] [1764831849.262258568] [AutoDrive]: Left and right centering deviation: 0.46
[INFO] [1764831849.264602851] [AutoDrive]: Initial error calculated: 0.32
```

Fill in `initial_error` in

`~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/config/auto_drive_node.yaml`

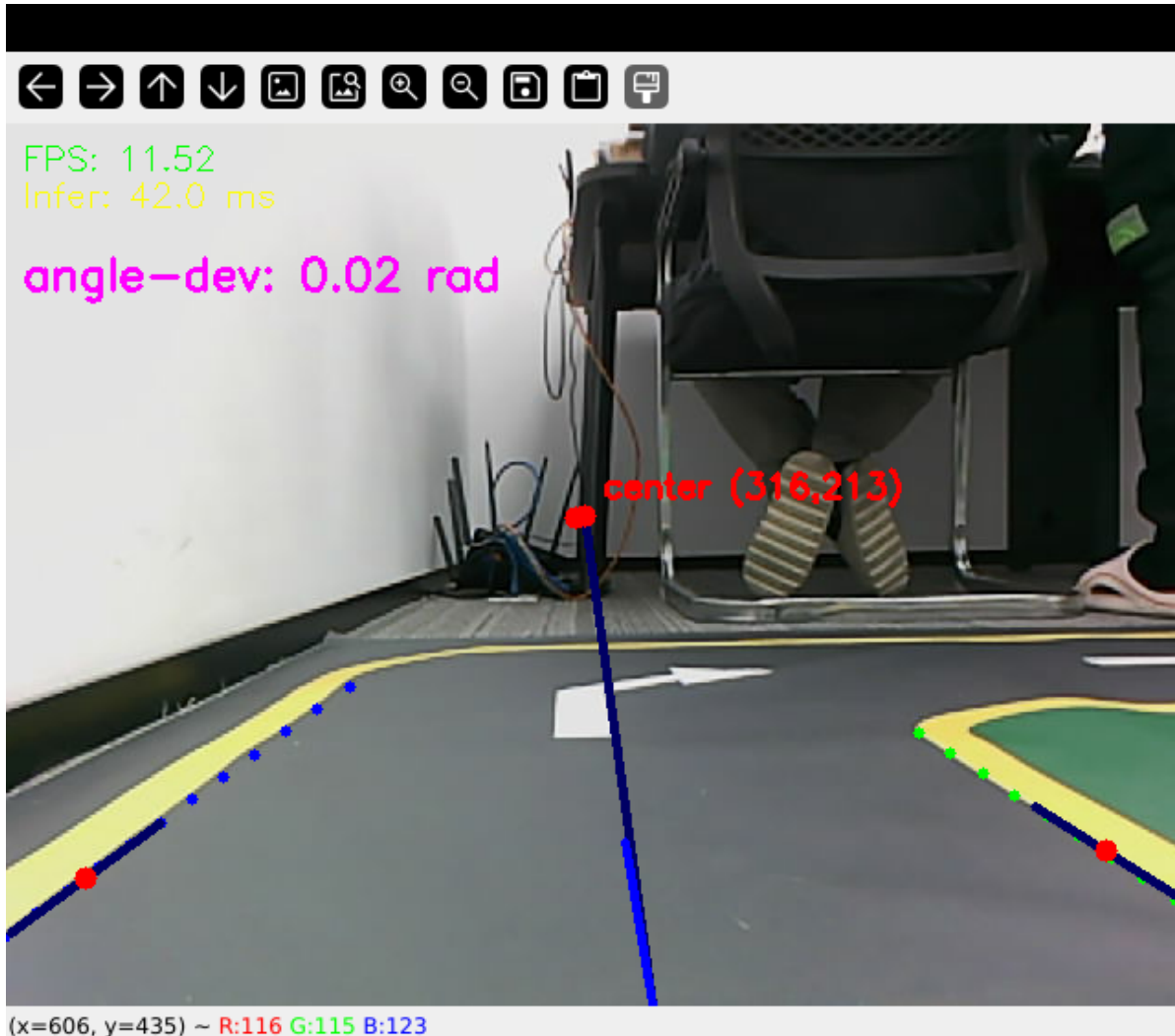


After modifying the parameters, recompile. The next run will use the modified parameters.

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/  
colcon build --packages-select auto_drive
```

5.2 Preview after calibration

After calibration, restart the program. You can see that the lane lines match the road.



6. Lane Line Detection Principle and Source Code Analysis

6.1 Lane Line Detection Principle

1. Lane line detection first uses the YOLOV11 object detection model to detect the distribution area of lane lines.

[!NOTE]

- If you need to train a custom lane line detection model, refer to Chapter 4, Model Training Tutorial.

2. Then, using Hough line detection, the longest line segment in the detection box is found. By judging the slope of the line, the left and right lane lines are divided.

[!TIP]

- Lane line detection can also be implemented using the YOLOv11 segmentation model and OBB rotation detection model, or other types of recognition models, but in actual tests on the Jetson device, the object detection model has a faster inference speed and better real-time performance.

The YOLOv11 object detection model can only detect the distribution of lane lines, but cannot directly distinguish between left and right lane lines because their features are extremely similar. If the data is labeled as left and right lane lines during the data annotation stage, the probability of misclassification during detection will be very high. Therefore, this study uses a combination of YOLOv11 and OpenCV Hough detection to comprehensively determine left and right lane lines.

6.2 Lane Line Detection Source Code

- YOLOv11 Inference Source Code:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/utils/yolov11_infer.py
```

- auto_drive.py Main Program Source Code:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/auto_drive.py
```

6.2.1 YOLOv11 Model Inference Source Code

- In yolov11_infer.py, YOLO_MODEL is the base class for YOLO detection, used to extend different YOLO recognition model programs.

```
class YOLO_MODEL(ABC):
    '''模型基类Model base class'''
    def __init__(self):
        self.yolo_model=None
    @abstractmethod
    def get_infer_result(self,source):
        """返回模型识别结果Return the model recognition result"""
        pass
    @abstractmethod
    def init_yolo_engine(self):
        """加载模型Load model"""
        pass
```

- Lane_Detect is a subclass of YOLO_MODEL and contains various methods for lane detection.

```
class Lane_Detect(YOLO_MODEL):
    """车道线检测模型Lane line detection model"""
    def __init__(self,config_file_path:str,
                 midline_file_path:str):
        with open(config_file_path, "r") as file:
            full_config = yaml.safe_load(file)
            self.config_param = full_config.get("line_detection", {})
        with open(midline_file_path, "r") as file:
            self.point_config = yaml.safe_load(file)
    -----
```

- `get_infer_result` is a method for obtaining lane detection results from inference images.

```
def get_infer_result(self, source, stream: bool = True):
    '''获取单帧图像的推理结果Obtain the reasoning result of a single frame image'''
    result =
self.yolo_model(source, stream=stream, conf=self.conf, iou=self.iou, rect=self.rect,
half=self.half,
device=self.device, batch=self.batch, max_det=self.max_det, augment=self.augment,
verbose=self.verbose, classes=self.classes)

    return result
```

- The `classify_lane_direction` method uses the slope of the longest straight line detected by Hough from the YOLO bounding box to help determine the left and right lane lines within the YOLO bounding box.

```
def classify_lane_direction(self, frame, box, debug=False):
    """
    霍夫检测判断车道线方向Hough detection determines the direction of lane lines
    """

    xmin, ymin, xmax, ymax = map(int, box)

    roi = frame[ymin:ymax, xmin:xmax]
    if roi.size == 0:
        return None

    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150)

    lines = cv2.HoughLinesP(
        edges, 1, np.pi / 180,
        threshold=30,
        minLineLength=20,
        maxLineGap=10
    )

    if lines is None:
        return None

    #Filter out horizontal and vertical lines
    filtered = []
    for line in lines:
        x1, y1, x2, y2 = line[0]
        dx = x2 - x1
        dy = y2 - y1

        if dx == 0:
            continue

        slope = dy / dx

        if abs(slope) < 0.03: # 太水平Too level
            continue
        if abs(slope) > 10: # 太垂直Too vertical
            continue
```

```

        filtered.append(line)

    if len(filtered) == 0:
        return None

    longest = max(
        filtered,
        key=lambda l: math.hypot(l[0][2]-l[0][0], l[0][3]-l[0][1])
    )

    x1, y1, x2, y2 = longest[0]

    if debug:
        px1, py1 = xmin + x1, ymin + y1
        px2, py2 = xmin + x2, ymin + y2
        cv2.line(frame, (px1, py1), (px2, py2), (0,255,0), 2)

    dx = x2 - x1
    dy = y2 - y1
    slope = dy / dx
    if slope < 0:
        return "left"
    else:
        return "right"

```

- The `__draw_dotted_line` method is used to draw the identified bounding boxes as visual lane lines (dashed line segments).

```

def __draw_dotted_line(self, image, start_point, end_point, color=(255, 0, 0),
    thickness=3, dot_spacing=20):
    """绘制由点组成的虚线段Draw dotted line segments composed of points"""

    # 计算两点间距离和方向Calculate the distance and direction between two
    points
    dx = end_point[0] - start_point[0]
    dy = end_point[1] - start_point[1]
    distance = np.sqrt(dx**2 + dy**2)

    num_dots = int(distance / dot_spacing)
    points = []
    for i in range(num_dots + 1):
        t = i / num_dots if num_dots > 0 else 0
        x = int(start_point[0] + t * dx)
        y = int(start_point[1] + t * dy)
        points.append((x, y))
    for point in points:
        cv2.circle(image, point, thickness, color, -1)
    return image

```

6.2.2 Main Program for Lane Line Detection Processing

- `image_callback` is a callback function that subscribes to camera topics. It receives real-time camera images, converts them into OpenCV format images, and pushes them into the buffer queue of the `Lane_Detect` class.

```
def image_callback(self, msg: Image)->None:
    '''ros图像回调函数,将ROS图像转换为OpenCV格式,加入处理队列中
    The ROS image callback function converts the ROS image to OpenCV format
    and adds it to the processing queue.
    '''
    cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    self.lane_detect.put_image(cv_image.copy())
```

- The `handle_lane` method is responsible for retrieving image frames from the cache queue of the `Lane_Detect` class and processing them. The processing flow is as follows: 1. Retrieve image frames from the queue → 2. Model inference to obtain lane detection results → 3. Classify left and right lane candidate boxes → 4. Filter the optimal left and right lane boxes → 5. Visualize the results → 6. Loop and wait for the next frame

The procedure for classifying left and right lane lines is as follows:

```
for res in results:
    boxes = res.boxes # All detection boxes in a single frame (xyxy format:
xmin, ymin, xmax, ymax)
    if self.DEBUG_MODE: frame = res.plot() # Debug mode: Draw all detection
boxes onto the image
    if boxes is not None:
        for i in range(len(boxes)):
            box_coords = boxes.xyxy[i].tolist() # Coordinates of a single
detection box (top left corner + bottom right corner)
            # Key: Call the classification function to determine whether the box
belongs to the left or right lane.
            lane = self.lane_detect.classify_lane_direction(frame, box_coords,
debug=self.DEBUG_MODE)
            if lane == "left":
                left_candidates.append(box_coords) # Add left lane candidate
            elif lane == "right":
                right_candidates.append(box_coords) # Add right lane candidate
def handle_lane(self)->None:
    '''处理车道线识别的检测结果Processing lane line recognition detection results'''

    while True:
        try:
            frame = self.lane_detect.image_queue.get(timeout=0.1)
        except queue.Empty:
            continue
        infer_start = time.time()
        results =self.lane_detect.get_infer_result(frame, stream=True)
        #初始化默认值Initialize default values
        left_box = None
        right_box = None
        left_candidates = []
        right_candidates = []

        for res in results:
            boxes = res.boxes # 检测对象边界框Boxes object for bounding box
outputs
            if self.DEBUG_MODE: frame = res.plot()
            if boxes is not None:
                for i in range(len(boxes)):
```

```

        box_coords = boxes.xyxy[i].tolist() # 获取边界框坐标
        (xyxy格式: xmin, ymin, xmax, ymax) Get the bounding box coordinates (xyxy format:
        xmin, ymin, xmax, ymax)

    lane=self.lane_detect.classify_lane_direction(frame,box_coords,debug=self.DEBUG
    _MODE)

        if lane=="left":
            left_candidates.append(box_coords)
        elif lane=="right":
            right_candidates.append(box_coords)

    if len(left_candidates)>0 :
        left_box = max(left_candidates, key=lambda box: box[3]) # 选择最靠近底
        部的边界框 (ymax最大) Select the bounding box closest to the bottom (with the
        largest ymax).
        if len(right_candidates)>0:
            right_box = max(right_candidates, key=lambda box: ((box[2] -
            box[0])**2 + (box[3] - box[1])**2)**0.5)

    frame=self.__handle_lane_detect_result(left_box,right_box,frame)
    infer_end = time.time()

    self.frame_count_1 += 1
    curr_time = time.time()
    time_diff = curr_time - self.prev_time_1
    if time_diff >= 1.0:
        self.fps_1 = self.frame_count_1 / time_diff
        self.frame_count_1 = 0
        self.prev_time_1 = curr_time
    cv2.putText(frame,f"FPS: {self.fps_1:.2f}",(10,
    25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)

    infer_time = (infer_end - infer_start) * 1000
    cv2.putText(frame, f"Infer: {infer_time:.1f} ms", (10,
    45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)
    cv2.imshow("Auto-Drive Panel", frame)
    cv2.waitKey(1)

```

